SQL Herhaling Graduaat Programmeren

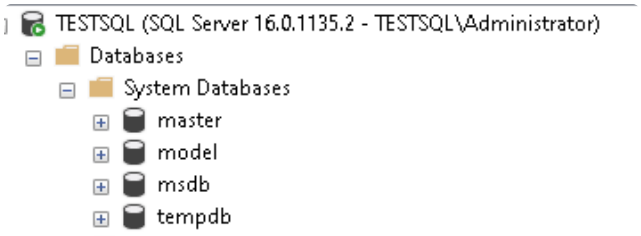
1. Voordat we starten met werken met een database

Elke database dient om gegevens in op te slaan. Wat die gegevens zijn, is afhankelijk van hoe de database opgezet is. In ons ERD Project voor Databases, maken we dus zelf een Database die met de tabellen uit ons ERD gevuld gaat worden.

Nu is het wel zo dat niet elke versie van SQL server dezelfde commandos aankan. Dit heeft te maken met functionaliteit die ingevoerd is in bepaalde versies.
Als men bijvoorbeeld SQL 2025 Server installeerd, kan dit ongewenste gevolgen hebben voor databases die afhankelijk zijn van bepaalde commando's uit een oudere code base.

Het is in SQL erg makkelijk om uit te vinden welke versie van SQL Server gebruikt wordt en wat de het compatibiliteits niveau (ditt is het versie niveau) is. (Compatibility Level).

De Standaard databasers die in een systeem aanwezig zijn zijn de volgende:



Deze worden NIET gebruikt om gegevens in op te slaan, maar om de werking van de server goed te laten verlopen.

Welke versie van de Database gebruikt wordt:

```
SELECT @@VERSION; --Dit geeft de versie van de database server terug
-- voorbeeld in Lab
'Microsoft SQL Server 2022 (RTM-GDR) (KB5046861) - 16.0.1135.2 (X64)
Oct 18 2024 15:31:58   Copyright (C) 2022 Microsoft Corporation
Standard Edition (64-bit) on Windows Server 2022 Datacenter 10.0 <X64>
(Build 20348: ) (Hypervisor)'
```

Dit is dus een SQL 2022 Standaard Server.

Het Compatibility Level:

```
SELECT compatibility_level
FROM sys.databases
WHERE [name] = 'DATABASENAAM'; --Master kan hier ook
-- Resultante in LAB:
'compatibility_level'
'160'
```

Een overzicht van dezew Compatibility levels kan gevonden worden op deze website:

- https://learn.microsoft.com/en-us/sql/t-sql/statements/alter-database-transact-sql-compatibility-level?view=sql-server-ver17

2. Create\Drop Database

Een database aanmaken:

```
create database DATABASENAAM;
-- VOORBEELD
CREATE DATABASE GradProg;
GO
```

De GO on het commando geeft aan dat het systeem dit commando ook meteen moet uitvoeren. Dat kan problemen met commando's die afhankelijk zijn van het aangemaakte item voorkomen.

Een veel voorkomende situatie bij het aanmaken van een database is het feit dat er een grootte van de database opgegeven moet worden. Dit maakt de query dan wat lastiger en wel om 2 redenen:

- het bestandsnaam moet aangegeven worden
- het juiste pad moet aangegeven worden.

De query moet dan als volgt opgezet worden:

```
create database GradProg
on primary
(    name = 'gradprog1',
     filename = 'C:\Program Files\Microsoft SQL
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\gradprog1.mdf',-- dit is dus het
volledige pad naar het bestand
     size = 10 MB);--de grootte van de database, let wel er komt ook
nog wat anders bij kijken
```

Een Database verwijderen:

```
DROP DATABASE DATABASENAAM;
--VOORBEELD
DROP DATABASE GradProg;
```

Dit levert soms een probleem op. Je krijgt dan de volgende foutmelding:

```
SQL Error [3702] [S0003]: Cannot drop database "DATABASENAAM" because
it is currently in use.
```

In dat geval moet de volgende procedure gevolgd worden:

```
USE master;
GO

ALTER DATABASE DATABASENAAM SET  SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO

USE master;
GO

DROP DATABASE DATABASENAAM;
GO
```

Het tweede commando zorgt ervoor dat de database geen openstaande verbindingen meer heeft.

3. Create\Drop Schema

Een schema is een soort Organisatorische eenheid waarmee we het volgende kunnen bereiken:

- Combineren van objecten als 1 geheel
- Objecten kunnen dezelfde naam hebben in verschillende schemas
- Het vergemakkelijkt het toekennen van permissies op de onderdelen van een database

Aanmaken Schema

```
CREATE SCHEMA Sprockets
    AUTHORIZATION Joe; -- hier wordt de gebruiker Joe and dit schema
toegevoegd
GO

CREATE TABLE Sprockets.NineProngs --direct een tabel (NineProngs)
aanmaken met een schema
(
   source INT,
   cost INT,
   partnumber INT
);
GO
```

```
GRANT SELECT
   ON SCHEMA::Sprockets TO Bob; --met dit GRANT statement krijgt Bob
toestemming om in het Schema Sprockets selecvt queries uit te voeren.
GO

DENY SELECT
   ON SCHEMA::Sprockets TO John; --Met dit DENY statement, wordt de
gebruiker John het recht ontnomen om in het Schema Sprockets een select
uit te voeren
GO

ALTER SCHEMA SCHEMANAAM TRANSFER SCHEMANAAM2.TABELNAAM; --Dit commando
verplaatst een tabel van het ene Schema naar het andere Schema
GO
```

Hier zien we dus ook het toekennen van rechten via het schema. Dit is door ons nog niet gedaan, maar is wel belangrijk in de praktijk.

Welke schema's bestaan in een database

Welke Schema's zijn er in een database:

```
select * from sys.schemas;
```

De volgende Schema's zijn standaard aanwezig in een Database:

- sys (systeem tabellen)
- dbo (database owner)
- guest (een gast schema)

Verwijderen Schema

Een Schema verwijderen gaat via de volgende code:

```
DROP SCHEMA SCHEMANAAM;
GO
```

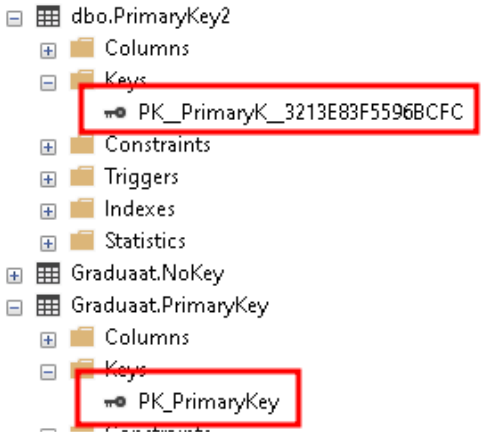
4. Create\Alter\Drop Table

Aanmaken Tabel

Het aanmaken van een tabel in SQL Server:

```
CREATE TABLE SCHEMANAAM.TABELNAAM
(
     KOLOM1 int IDENTITY(1,1),
     KOLOM2 varchar(100),
     KOLOM3 varchar(100),
     CONSTRAINT PK_TABELNAAM PRIMARY KEY (KOLOM1)
);
```

OPMERKING: Het aanmaken van een Primary Key is misshien omslachtig op deze manier, maar het geeft on de mogelijkheid om de naam van de Primary Key zelf te bepalen.
Onderstaand screenshot geeft aan wat er gebeurd als we de Primary Key declaratie in-line done met de juiste kolom:



De bovenste is gegenereerd in-line met de juiste kolom, de onderste met bovenstaand commando.

```
create table Graduaat.PrimaryKey2
(
     id int not null identity(1,1) Primary Key, --in-line PK declaratie
     naam varchar(45) not null
);
```

Wijzigen van een Tabel

Het toevoegen van een extra kolom of constraint aan een bestaande tabel is mogelijk door middel van de volgende commando's:

```
-- een of meerdere kolommen toevoegen aan een tabel
ALTER TABLE SCHEMANAAM.TABELNAAM
     ADD KOLOMNAAM DATATYPE;

-- een CONSTRAINT toevoegen aan een tabel
ALTER TABLE SCHEMANAAM.TABELNAAM
     ADD CONSTRAINT FK_FKTABELNAAM_PKTABELNAAM
```

```
    FOREIGN KEY (KOLOMNAAM)
    REFERENCES SCHEMANAAM.PKTABELNAAM (IDKOLOM);
```

Het wijzigen van een datatype in een Tabel kan met het volgende commando:

```
ALTER TABLE SCHEMANAAM.TABELNAAM
 ALTER COLUMN KOLOMNAAM NIEUWDATATYPE;
```

Constraints kunnen NIET gewijzigd worden. Moet er een aanpassing plaatsvinden, dan is de enige mogelijkheid:

- Constraint verwijderen
- Constraint opnieuw aanmaken

Wijzigen van NULL naar NOT NULL voor een kolom of DEFAULT waarde

Een vaak voorkomend probleem is dat een kolom in de Tabel NULL waarden mag bevatten. Als dit aangepast moet worden, is dat vaak vervelend, niet moeilijk, maar het kan meestal niet in 1 stap.
Neem de tabel die met de volgende code is aangemaakt:

```
create table Player (
    id int identity(1,1) unique,
    first_name varchar(50) null,
    last_name varchar(50) null,
    email nvarchar(100) null,
    constraint PK_Player primary key (id)
);
```

De kolommen first_name, last_name en email mogen nu NULL waarden bevatten. Als er nu al data in de tabel is opgeslagen:

```
insert into Player (first_name,last_name) Values ('Corne','Snoek');
```

Dan is het niet meer mogelijk om de kolom 'email' van NULL naar NOT NULL om te zetten zonder dat we dit gecorrigeerd hebben.

```
-- Dus eerst toevoegen van ontbrekende data
update player set email = 'corne.snoek@mail.de' where id = 1;
-- en daarna kolom aanpassen
alter table Player
alter column email nvarchar(100) not null;
```

Verwijderen Tabel

Het verwijderen van een tabel gebeurd met het volgende commando:

```
DROP TABLE SCHEMANAAM.TABELNAAM;
```

Het achteraf verwijderen van een kolom of constraint gebeurt met de volgende commando's:

```
-- Kolom verwijderen
ALTER TABLE SCHEMANAAM.TABELNAAM
     DROP COLUMN KOLOMNAAM DATATYPE;

--een Constraint verwijderen
ALTER TABLE SCHEMANAAM.TABELNAAM
     DROP CONSTRAINT PK_PKTABELNAAM;
ALTER TABLE SCHEMANAAM.TABELNAAM
     DROP CONSTRAINT FK_FKTABELNAAM_PKTABELNAAM;
```

DataTypes

Op deze website is een compleet overzicht van de datatypes in SQL Server te vinden:

- https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver17
- https://www.w3schools.com/sql/sql_datatypes.asp

Een kort overzicht:

- INT:
  - Geheel getal, standaard datatype voor een Primary Key/Foreign Key/Numerieke waarden waar we mee willen rekenen
- CHAR(x)/NCHAR(x):
  - hier moet altijd een lengte van de Character string opgegeven worden. Is een tekststring met de lengte van x (mocht de string korter zijn dan wordt het resterende aantal characters aangevuld met spaties!!!)
- VARCHAR(x)/NVARCHAR(x):
  - ook hier moet een lengte van de string opgegeven worden. Het grote verschil met CHAR is dat hier de resterende ruimte NIET met spaties wordt opgevuld.
  - NVARCHAR(x) is de Unicode versie van VARCHAR(x)
- NUMERIC(m,d)\DECIMAL(m,d):
  - Beide zijn getallen met een vaststaand aantal getallen achter de komma. met M wordt het aantal getallen opgegeven, met D het aantal posities na de komma.

- TINYINT/SMALLINT/BIGINT:
  - gehele getallen in verschillende groottes
- BIT:
  - Geheel getal 1, 0 of NULL
- FLOAT(n)\REAL:
  - voor getallen met een komma. de n geeft de precisie weer van het getal. (Bij n<25 wordt een FLOAT een REAL).
- SMALLMONEY/MONEY:
  - een datatype dat een Monetaire waarde bevat.
- DATE:
  - alleen de datum, standaardformaat "YYYY-MM-DD". Dit kan gewijzigd worden
- TIME(x):
  - alleen de tijd, standaardformaat "hh:mm:ss[.nnnnnnn]". Met de x wordt de precisie van de nanoseconden aangegeven (0-7)
- DATETIME/DATETIME2(x)/SMALLDATETIME/DATETIMEOFFSET:
  - datum en tijdswaarden. De laatste geeft ook nog de offset ten opzichte van UTC aan.
  - (UTC = Universal Time Coordinated. De Tijd in Greenwich(UK) in de winter. Kent geen zomer- of wintertijd).
  - DATETIME moet vermeden worden, gaat verwijderd worden.
  - Een overzicht van alle manieren waarop we een datum/tijd kunnen formateren:
    - https://database.guide/list-of-date-formats-available-with-convert-in-sql-server/
    - https://www.mssqltips.com/sqlservertip/2655/format-sql-server-dates-with-format-function/
- TEXT
  - Gaat in de toekomst verwijderd worden. NIET meer gebruiken. Alternatief is varchar(max) of nvarchar(max).

CONSTRAINTS

Constraints zijn beperkingen of voorwaarden op een database. Bijvoorbeeld:

- Velden moeten wel of niet ingevuld zijn (NULL/NOT NULL)
- Velden moeten Uniek zijn (of een combinatie van velden) (**Constraint UK_Medewerker_id UNIQUE (id)**)
- Velden zijn mogelijk een
  - PRIMARY KEY
    - **Constraint PK_TABELNAAM PRIMARY KEY (id)**
    - vaak in combinatie met IDENTITY(1,1) op de kolom van de id.

    - FOREIGN KEY
        - Geeft relatie tussen 2 Tabellen weer (altijd van de refererende Tabel naar de tabel met de Primary KEY)
        - **Constraint FK_TABELFK_TABELPK FOREIGN KEY (kolomid) REFERENCES TABELNAAMPK (id)**
        - Bij foreign keys dienen we ook nog de volgende instellingen goed te zetten:
            - DTC: ON DELETE CASCADE --> als de data van de Primaire Sleutel verwijderd wordt, worden ook alle rijen met deze data verwijderd.
            - DTR: ON DELETE RESTRICT (Is voor SQL Server de standaard! en hoeft dus niet gedeclareerd te worden. Sterker nog, dit geeft een foutmelding.) --> Zolang er nog rijen met deze data in tabellen bestaat, mag dit niet verwijderd worden.
            - DTN: ON DELETE SET NULL --> De data die de Foreign Key bevat wordt op NULL gezet.
- Velden mogen alleen een bepaalde waarde bevatten
    - **CONSTRAINT CK_TABELNAAM_KOLOMNAAM CHECK (KOLOMNAAM = 'X' or KOLOMNAAM = 'Y')**
    - De check can ook over meedere kolommen gedaan worden (KOLOMNAAM1 = 'X' OR KOLOMNAAM2 = 'Y') in dit geval of de een of de ander
    - de Check kan over meerdere kolommen gedaan worden en beide kolommen moeten de juiste waarde bevatten (KOLOMNAAM = 'X' AND KOLOMNAAM2 = 'Y'). In dit geval zowel de een als de ander moet waar zijn.
- Velden krijgen een standaard waarde als ze niet ingevuld worden
    - **Constraint DF_TABELNAAM_KOLOMNAAM DEFAULT 'WAARDE'**
    - Dit wordt vaak gezet bij NOT NULL kolommen, om te voorkomen dat een fout optreedt als deze data vergeten mocht worden.
    - volgt direct na de definitie van de kolom en wordt dus niet met een komma van deze definitie gescheiden.

```
CREATE TABLE Les.Persoon
(
int IDENTITY(1,1),
familienaam varchar(100) NOT NULL,
voornaam varchar(100),
geslacht char,
postcode varchar(4),
gemeente varchar(100),
land varchar(100) CONSTRAINT DF_Persoon_land DEFAULT 'België', -
- geen komma, de Constraint hoort bij land
```

```
    CONSTRAINT PK_Persoon PRIMARY KEY (id)
    );
```

Functie Primary Key

De PRIMARY KEY constraint in SQL zorgt ervoor dat je:

1. **Rijen Data uniek kunt identificeren:** Zorg ervoor dat elke rij in een Tabel een unieke identificatie heeft. Daardoor kun je snel en efficient de data ophalen en individuele Rijen managen.
2. **Behoud van Dataintegriteit:** Voorkom het invoeren van dubbele records, zodat de nauwkeurigheid en consistentie van gegevens behouden blijft.
3. **Relaties tot stand brengen:** Creëer relaties tussen tabellen met behulp van externe sleutels die verwijzen naar de primaire sleutel van een andere tabel. Dit is van fundamenteel belang voor het definiëren van gegevensrelaties in een relationele database.
4. **Verbeter de queryprestaties:** Verbeter de queryprestaties door de primaire sleutelkolom(men) of indexen te gebruiken als basis voor het zoeken, samenvoegen en filteren van gegevens.

Kolomconstraints:

- Alleen geldig voor 1 Kolom
  - data type
  - DEFAULT
  - NULL/NOT NULL **Tabelconstraints:**
- PK
- FK
- UNIQUE
- CHECK

In welke volgorde Tabellen aanmaken/verwijderen

Het is belangrijk om in de juiste volgorde tabellen aan te maken of te verwijderen.
Een tabel met een Foreign Key kan niet aangemaakt worden als de tabel waar deze key naar verwijst nog niet bestaat. Omgekeerd kun je dus geen tabel verwijderen waar nog een verwijzing via een Foreign Key naar bestaat.

DUS:

1. Tabellen zonder Foreign Keys
2. Tabellen met een FK

Dit lijkt een open deur, maar je kan dus beginnen met het aanmaken van Tabellen met een Foreign Key verwijzing zodra de tabel waarnaar verwezen wordt, ook daadwerkelijk bestaat.

Voor verwijderen geldt dus precies het omgekeerde, je kunt geen tabel verwijderen zolang er nog een tabel met een Foreign Key verwijzing bestaat! Zelfs bij een Cascade of NULL CONSTRAINT werkt dit niet. In dat geval:

1. Tabellen met een FK
2. Tabellen zonder een FK

5. Data verwijderen\toevoegen\wijzigen uit\aan\in een tabel

Inhoud van een Tabel verwijderen

De inhoud van een Tabel verwijderen kan op 2 manieren:

1. Truncate
   - Data wordt verwijderd
   - Identity start weer bij de ingestelde startwaarde
   - LET OP, bij een truncate wordt altijd de GEHELE INHOUD van een tabel verwijderd
2. Delete
   - Data wordt verwijderd
   - Identity blijft op laatste waarde staan
   - door middel van een WHERE clausule kan een deel van de data verwijderd worden.

Ook hier is de volgorde in principe:

1. Data met de FK eerst verwijderen
2. Dan data waarnaar verwezen wordt.

```
TRUNCATE TABLE SCHEMANAAM.TABELNAAM;
DELETE FROM SCHEMANAAM.TABELNAAM;
DELETE FROM SCHEMANAAM.TABELNAAM WHERE ......;
```

Data aan een tabel toevoegen

Hiervoor zijn verschillende methoden:

1. Insert into...
2. Select into ....

Insert Into

Syntax Insert Into:

```
INSERT INTO SCHEMANAAM.TABELNAAM
     (KOLOMNAAM1, KOLOMNAAM2, KOLOMNAAM3 .. KOLOMNAAMx)
     VALUES (WAARDE1, WAARDE2, WAARDE3 .. WAARDEx);
```

Belangrijk bij deze query zijn de volgende punten:

- ALTIJD dezelfde hoeveelheid waarden definieren als dat er kolommen gedefinieerd zijn. (Dit gaat anders echt fout!)
- Zorg dat de type van de Data overeenkomen
- Dat de volgorde van de waarden overeenkomt met de volgorde van de kolommen
- niet vermelde kolommen krijgen NULL-waarde, TENZIJ
    - De vergeten kolom geen NULL waardes toestaat, krijgen we een foutmelding.
    - de kolom een Identity kolom is, dan mag deze waarde niet ingegeven worden!
    - De kolom een default waarde krijgt, dan komt deze waarde in de kolom te staan en geen NULL
- NULL mag als waarde meegegeven worden

Van de Insert into is nog een tweede variant beschikbaar. Daarbij worden geen waarden direkt opgegeven, maar opgehaald met een select statement.
Bij voorbeeld:

```
INSERT INTO SCHEMANAAM.TABELNAAM
     (KOLOMNAAM1, KOLOMNAAM2, KOLOMNAAM3 .. KOLOMNAAMx)
SELECT WAARDE1, WAARDE2, WAARDE3 .. WAARDEx
     FROM SCHEMANAAM.TABELNAAM2 WHERE ...;
```

Ook hier moeten we oppassen dat we dezelfde hoeveelheid waarden meegeven als dat er als kolommen zijn opgegeven.
Met dit statement kunnen we in 1 keer meerdere rijen data aan een tabel toevoegen.

Gevaarlijke variant van dit commando:

```
INSERT INTO SCHEMANAAM.TABELNAAM
     SELECT * FROM SCHEMANAAM.TABELNAAM2 WHERE ...;
```

In dit geval kan het voorkomen dat er te veel of te weinig kolommen geselecteerd worden. Naast het feit dat het hier onduidelijk is wat er precies gebeurd, kan het tot fouten leiden.

Als nu iemand een van de tabellen aanpast door een kolom toe te voegen of weg te halen, zal dit een probleem op leveren.

Select Into

De tweede variant is het SELECT INTO statement.
Door middel van dit statement kunnen we een nieuwe tabel aanmaken met de kolommen die we geselecteerd hebben van een bestaande tabel.

```
-- alle kolommen van een tabel
SELECT *
     INTO SCHEMANAAM.NIEUWETABELNAAM [IN DATABASENAAM]
     FROM SCHEMANAAM.OUDETABELNAAM
     WHERE ...;

-- bepaalde kolommen van een tabel.
SELECT KOLOMNAAM1, KOLOMNAAM2, KOLOMNAAM3 .. KOLOMNAAMx
     INTO SCHEMANAAM.NIEUWETABELNAAM [IN DATABASENAAM]
     FROM SCHEMANAAM.OUDETABELNAAM
     WHERE ...;
```

Inhoud van een tabel wijzigen

Het wijzigen van data in een tabel gebeurt met het UPDATE-statement. Dit statement ziet er als volgt uit:

```
UPDATE SCHEMANAAM.TABELNAAM
     SET KOLOMNAAM1 = WAARDE1,
          KOLOMNAAM2 = WAARDE2,
          KOLOMNAAM3 = WAARDE3,
     ...
WHERE ...;
```

Alternatieve Sleutel Of Index

Een index heeft als functie dat de gegevens sneller uit een database op te halen zijn.
Het aanmaken van een Index heeft implicaties voor de database en het wordt dus aangeraden om alleen een Index op die kolommen te plaatsen waar veel naar gezocht wordt.
De Index is een manier om een Alternative Key in te geven.

Aanmaken Index

Het aanmaken van een Index gebeurt met de volgende syntax:

```
CREATE INDEX IND_TABELNAAM_KOLOMNAAM
ON SCHEMANAAM.TABELNAAM (KOLOMNAAM1,(KOLOMNAAM2, ...));
```

Men kan ook een Index aanleggen die alleen unieke waarden toestaat. In dat geval is de syntax iets anders:

```
CREATE UNIQUE INDEX IND_TABELNAAM_KOLOMNAAM
ON SCHEMANAAM.TABELNAAM (KOLOMNAAM1,(KOLOMNAAM2, ...));
```

Verwijderen Index

Met de volgende syntax wordt een Index uit het systeem verwijderd:

```
DROP INDEX SCHEMANAAM.TABELNAAM.IND_TABELNAAM_KOLOMNAAM;
```

6. SELECT Statements

Het eenvoudigste voorbeeld van een select statement is het volgende statement:

```
SELECT *
FROM SCHEMANAAM.TABELNAAM;
```

De * geeft aan dat ALLE Kolommen in een tabel geselecteerd moeten worden. Dit kan potentieel problemen opleveren als de tabel gewijzigd wordt en er dan in plaats van 3 in eens 4 kolommen terug komen. Dit geeft in veel productie omgevingen problemen.

Een iets andere variant daarop is deze Query:

```
SELECT KOLOM1, KOLOM2, KOLOM3, ... KOLOMX
FROM SCHEMANAAM.TABELNAAM;
```

Deze query zal altijd alleen maar de gevraagde kolommen weergeven en er zal dus bij een wijziging van de tabel geen wijziging in het resultaat van deze query optreden.

Select .. Where statements

Om gerichter data terug te kunnen krijgen wordt de WHERE Clausule gebruikt. Hiermee kunnen we de data die teruggegeven wordt tot een vooraf gedefinieerde set beperken. Bijvoorbeeld als we in een tabel een kolom met de datum hebben, dan kunnen we bijvoorbeeld kijken wat er op 7 december 2025 in deze tabel voor data beschikbaar is. Deze query ziet er als volgt uit:

```
SELECT * FROM SCHEMANAAM.TABELNAAM
WHERE KOLOMDATUM = '2025-12-07';
```

Voor de Where condities kunnen we de volgende operatoren gebruiken:

- = (is gelijk aan)
- > (is groter dan)
- >= (is groter of gelijk aan)
- < (is kleiner dan)
- <= (is kleiner of gelijk aan)
- <> (is niet gelijk aan)
- != (is niet gelijk aan)
- (NOT) IN (de waarde van de kolom ligt (niet) in de lijst met waardes die hier gegeven is)
- (NOT) BETWEEN (de waarde van de kolom ligt (niet) tussen beide waarden (inclusief deze beide waarden). Dit is ook te schrijven met 2 condities >= and <=)
- (NOT) LIKE (een text gebaseerde operator, Lijkt (niet) op. Let op, dit kan afhankelijk van de instellingen case sensitive zijn)
  - hierbij kunnen wildcard characters gebruikt worden, te weten:
    - % --> 0, 1 of meerdere willekeurige tekens
    - _ --> 1 Willekeurig teken
  - In sommige gevallen moeten characters niet in hun functie gebruikt worden. Dan worden deze door een ESCAPE character ontdaan van hun eigenlijke functie. Bijvoorbeeld het zoeken op een '_' zou in SQL betekenen zoeken op 1 willekeurig character, niet zoeken op een underscore. Door dit als volgt op te schrijven:
    ```
    SELECT *
    FROM SCHEMANAAM.TABELNAAM
    WHERE KOLOMNAAM LIKE '%\_%' ESCAPE '\';
    ```
    Is dit te voorkomen. Nu wordt specifiek voor het '_' de functie weggenomen en als een normaal character beschouwd.
  - Dit ESCAPE character wordt gezet middels de ESCAPE clausule, en kan dus door elk character worden vervangen.
- IS (NOT) NULL (Heeft (wel) geen waarde)
  - let hierbij op, dit wordt nooit als = NULL geschreven

In de WHERE clause kunnen we meerdere condities plaatsen, allen gescheiden door een van de woorden AND en OR.
De OR is een of conditie, waarbij dus aan of de een of de ander voldaan moet worden.
De AND conditie is een conditie waarbij aan beide voorwaarden moet worden voldaan.

In de praktijk gaat het gebruik van AND en OR nog wel eens fout.
Men kan door middel van accolades hier consistentie in de afgevraagde data krijgen.
Gegeven is de volgende tabel:

```
CREATE TABLE Sales.OrderDetails(
     orderid int NOT NULL,
     productid int NOT NULL,
     unitprice money NOT NULL,
     qty smallint NOT NULL,
     discount numeric(4, 3) NOT NULL,
 CONSTRAINT PK_OrderDetails PRIMARY KEY CLUSTERED
(orderid ASC, productid ASC),
CONSTRAINT FK_OrderDetails_Orders FOREIGN KEY(orderid)
REFERENCES Sales.Orders (orderid),
CONSTRAINT FK_OrderDetails_Products FOREIGN KEY(productid)
REFERENCES Production.Products (productid);
```

In deze tabel bevindt zich de volgende data:

	orderid	productid	unitprice	qty	discount
1	10248	11	14.00	12	0.000
2	10248	42	9.80	10	0.000
3	10248	72	34.80	5	0.000
4	10249	14	18.60	9	0.000
5	10249	51	42.40	40	0.000
6	10250	41	7.70	10	0.000
7	10250	51	42.40	35	0.150
8	10250	65	16.80	15	0.150
9	10251	22	16.80	6	0.050
10	10251	57	15.60	15	0.050
11	10251	65	16.80	20	0.000
12	10252	20	64.80	40	0.050
13	10252	33	2.00	25	0.050
14	10252	60	27.20	40	0.000
15	10253	31	10.00	20	0.000
16	10253	39	14.40	42	0.000

Als de vraag nu is, vindt alle order regels waar het bedrag groter is dan 25 en de korting groter is dan 0.200, dan is de resulteren de query:

```
SELECT *
FROM sales.OrderDetails
WHERE unitprice > '25' AND discount > '0.200'
```

en de uitkomst:

	orderid	productid	unitprice	qty	discount
1	10260	62	39.40	15	0.250
2	10279	17	31.20	15	0.250
3	10284	27	35.10	15	0.250
4	10284	60	27.20	20	0.250
5	10298	59	44.00	30	0.250
6	10344	8	32.00	70	0.250
7	10369	56	30.40	18	0.250
8	10372	20	64.80	12	0.250
9	10372	38	210.80	40	0.250
10	10372	60	27.20	70	0.250
11	10372	72	27.80	42	0.250
12	10429	63	35.10	35	0.250
13	10431	17	31.20	50	0.250
14	10536	12	38.00	15	0.250
15	10536	60	34.00	35	0.250
16	10560	62	49.30	15	0.250

Veranderen we de query nu naar deze:

```
SELECT *
FROM sales.OrderDetails
WHERE unitprice > '25' AND discount > '0.200' OR qty = 1
```

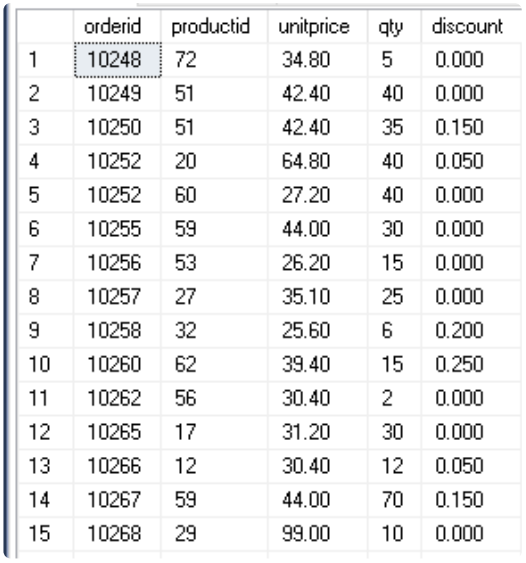
Dan kan er verwarring ontstaan over de uitkomst.

	orderid	productid	unitprice	qty	discount
1	10259	37	20.80	1	0.000
2	10260	62	39.40	15	0.250
3	10279	17	31.20	15	0.250
4	10281	19	7.30	1	0.000
5	10284	27	35.10	15	0.250
6	10284	60	27.20	20	0.250
7	10298	59	44.00	30	0.250
8	10308	69	28.80	1	0.000
9	10344	8	32.00	70	0.250
10	10369	56	30.40	18	0.250
11	10372	20	64.80	12	0.250
12	10372	38	210.80	40	0.250
13	10372	60	27.20	70	0.250
14	10372	72	27.80	42	0.250
15	10429	63	35.10	35	0.250
16	10431	17	31.20	50	0.250

Schrijven we de query nu iets anders:

```
SELECT *
FROM sales.OrderDetails
WHERE unitprice > '25' OR qty = 1 AND discount > '0.200'
```

Dan is de uitkomst:

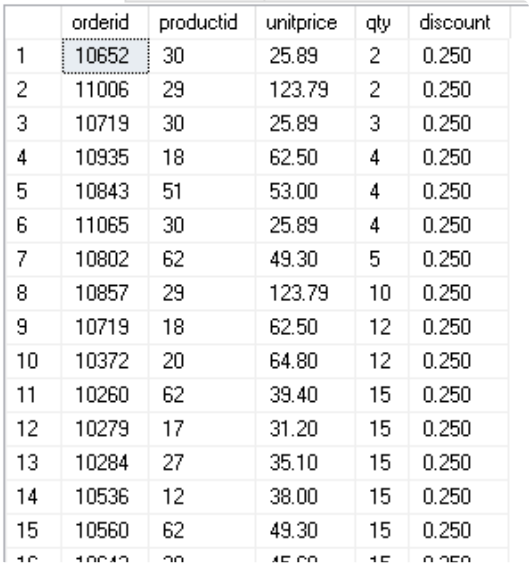


	orderid	productid	unitprice	qty	discount
1	10248	72	34.80	5	0.000
2	10249	51	42.40	40	0.000
3	10250	51	42.40	35	0.150
4	10252	20	64.80	40	0.050
5	10252	60	27.20	40	0.000
6	10255	59	44.00	30	0.000
7	10256	53	26.20	15	0.000
8	10257	27	35.10	25	0.000
9	10258	32	25.60	6	0.200
10	10260	62	39.40	15	0.250
11	10262	56	30.40	2	0.000
12	10265	17	31.20	30	0.000
13	10266	12	30.40	12	0.050
14	10267	59	44.00	70	0.150
15	10268	29	99.00	10	0.000

Dan krijgen we in eens alleen maar items waarvan er meer dan 5 verkocht zijn, de AND ligt nu tussen qty en discount en daarvoor bestaat geen combinatie. Met andere woorden, de uitkomst van onze query is niet eenduidig en we kunnen er dus niet vanuit gaan dat we dezelfde informatie gaan krijgen als we dit anders ingeven.
Door het gebruik van accolades, kunnen we de afloop van deze query op consistente wijze beinvloeden.

```
SELECT *
FROM sales.OrderDetails
WHERE (unitprice > '25' OR qty = 1) AND discount > '0.200'
```

In dit geval wordt altijd gezocht naar die producten waar de prijs groter is dan 25 en er een discount is van meer dan 20% of naar die producten, waar er maar 1 van verkocht is en de discount meer dan 20% is.
Resulterend in:



	orderid	productid	unitprice	qty	discount
1	10652	30	25.89	2	0.250
2	11006	29	123.79	2	0.250
3	10719	30	25.89	3	0.250
4	10935	18	62.50	4	0.250
5	10843	51	53.00	4	0.250
6	11065	30	25.89	4	0.250
7	10802	62	49.30	5	0.250
8	10857	29	123.79	10	0.250
9	10719	18	62.50	12	0.250
10	10372	20	64.80	12	0.250
11	10260	62	39.40	15	0.250
12	10279	17	31.20	15	0.250
13	10284	27	35.10	15	0.250
14	10536	12	38.00	15	0.250
15	10560	62	49.30	15	0.250

In dit geval zien we dus alleen maar items met een korting van meer dan 20%.

Het foutief of niet gebruiken van accolades kan zeer onbetrouwbare data opleveren en moet dus altijd goed in overweging genomen worden.

DISTINCT operator

De DISTINCT operator haalt de dubbele waardes uit de query. Let op dat dit altijd betrekking heeft op de gehele rij en niet alleen de kolom waar DISTINCT voor staat. Voorbeeld (op basis van de vorige tabel):

```
select distinct orderid
FROM sales.OrderDetails
```

geeft een lijst met unieke OrderIDs.

	orderid
1	10248
2	10249
3	10250
4	10251
5	10252
6	10253
7	10254

Anders dan deze query:

```
select distinct orderid, productid
FROM sales.OrderDetails
order by orderid asc, productid asc
```

Hier krijgen we meerdere keren hetzelfde orderID terug, maar de combinatie orderid en productid is uniek en wordt dus maar 1 keer getoond.

	orderid	productid
1	10248	11
2	10248	42
3	10248	72
4	10249	14
5	10249	51
6	10250	41
7	10250	51
8	10250	65
9	10251	22
10	10251	57
11	10251	65

NULL Waarden

NULL staat voor geen waarde en hiermee is erg slecht te werken. Verschillende functies kunnen ook erg slecht met deze waarden omgaan.

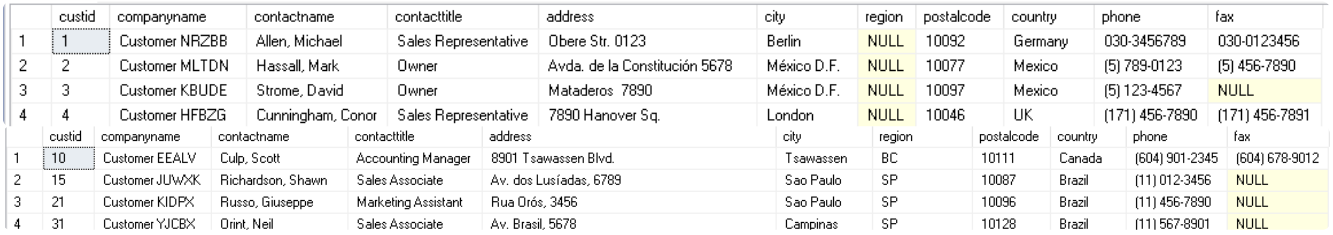
Als voorbeeld nemen we hier de volgende tabel:

```
CREATE TABLE Sales.Customers(
     custid int IDENTITY(1,1) NOT NULL,
     companyname nvarchar(40) NOT NULL,
     contactname nvarchar(30) NOT NULL,
```

```
    contacttitle nvarchar(30) NOT NULL,
    address nvarchar(60) NOT NULL,
    city nvarchar(15) NOT NULL,
    region nvarchar(15) NULL,
    postalcode nvarchar(10) NULL,
    country nvarchar(15) NOT NULL,
    phone nvarchar(24) NOT NULL,
    fax nvarchar(24) NULL,
 CONSTRAINT PK_Customers PRIMARY KEY CLUSTERED (custid ASC)
```

In deze tabel kunnen de kolommen region en fax potentieel NULL waarden bevatten. Een selectie van die klanten die net wel of geen Region waarde hebben kan door middel van volgende query:

```
SELECT *
FROM sales.customers
WHERE region IS (NOT) NULL
```

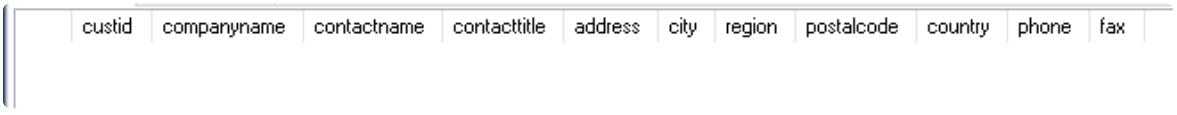


	custid	companyname	contactname	contacttitle	address	city	region	postalcode	country	phone	fax
1	1	Customer NRZBB	Allen, Michael	Sales Representative	Obere Str. 0123	Berlin	NULL	10092	Germany	030-3456789	030-0123456
2	2	Customer MLTDN	Hassall, Mark	Owner	Avda. de la Constitución 5678	México D.F.	NULL	10077	Mexico	(5) 789-0123	(5) 456-7890
3	3	Customer KBUDE	Strome, David	Owner	Mataderos 7890	México D.F.	NULL	10097	Mexico	(5) 123-4567	NULL
4	4	Customer HFBZG	Cunningham, Conor	Sales Representative	7890 Hanover Sq.	London	NULL	10046	UK	(171) 456-7890	(171) 456-7891

	custid	companyname	contactname	contacttitle	address	city	region	postalcode	country	phone	fax
1	10	Customer EEALV	Culp, Scott	Accounting Manager	8901 Tsawassen Blvd.	Tsawassen	BC	10111	Canada	(604) 901-2345	(604) 678-9012
2	15	Customer JUWXK	Richardson, Shawn	Sales Associate	Av. dos Lusíadas, 6789	Sao Paulo	SP	10087	Brazil	(11) 012-3456	NULL
3	21	Customer KIDPX	Russo, Giuseppe	Marketing Assistant	Rua Orós, 3456	Sao Paulo	SP	10096	Brazil	(11) 456-7890	NULL
4	31	Customer YJCBX	Orint, Neil	Sales Associate	Av. Brasil, 5678	Campinas	SP	10128	Brazil	(11) 567-8901	NULL

De volgende query werkt niet in dit geval:

```
SELECT *
FROM sales.customers
WHERE region <> null
```



custid	companyname	contactname	contacttitle	address	city	region	postalcode	country	phone	fax

Ook controles op kolommen met NULL waardes kunnen extreem lastig zijn. In oudere SQL versies was dit nog lastiger, dan in SQL 2022 en hoger. Daar worden de NULL waarden weggelaten uit de meeste operaties (hetgeen ook niet acceptabel kan zijn).

Ook string concatenaties kunnen fout gaan mits men de juiste functies gebruikt. Bijvoorbeeld bij deze query:

```
SELECT custid, companyname, address, postalcode, city, region,
countryFROM sales.customers
```

Krijgen we keurig alle data terug:

	custid	companyname	address	postalcode	city	region	country
1	1	Customer NRZBB	Obere Str. 0123	10092	Berlin	NULL	Germany
2	2	Customer MLTDN	Avda. de la Constitución 5678	10077	México D.F.	NULL	Mexico
3	3	Customer KBUDE	Mataderos 7890	10097	México D.F.	NULL	Mexico
4	4	Customer HFBZG	7890 Hanover Sq.	10046	London	NULL	UK
5	5	Customer HGVLZ	Berguvsvägen 5678	10112	Luleå	NULL	Sweden
6	6	Customer XHXJV	Forsterstr. 7890	10117	Mannheim	NULL	Germany
7	7	Customer QXVLA	2345, place Kléber	10089	Strasbourg	NULL	France
8	8	Customer QUHWH	C/ Araquil, 0123	10104	Madrid	NULL	Spain
9	9	Customer RTXGC	6789, rue des Bouchers	10105	Marseille	NULL	France
10	10	Customer EEALV	8901 Tsawassen Blvd.	10111	Tsawassen	BC	Canada
11	11	Customer UBHAU	Fauntleroy Circus 4567	10064	London	NULL	UK
12	12	Customer PSNMQ	Cerrito 3456	10057	Buenos Aires	NULL	Argentina
13	13	Customer VMLOG	Sierras de Granada 7890	10056	México D.F.	NULL	Mexico
14	14	Customer WNMAF	Hauptstr. 0123	10065	Bern	NULL	Switzerland
15	15	Customer JUWXK	Av. dos Lusíadas, 6789	10087	Sao Paulo	SP	Brazil

Maar dus ook de NULL waarden. In een query waar we het adres als 1 string willen terugkrijgen, kan dit (afhankelijk van hoe we dit doen) compleet fout gaan:

```
SELECT custid, (companyname+' '+ address+' '+ postalcode+' '+city+'
'+region+' '+country) as companyaddress
FROM sales.customers
```

De contactenatie door middel van + gaat helemaal de mist in:

	custid	companyaddress
1	1	NULL
2	2	NULL
3	3	NULL
4	4	NULL
5	5	NULL
6	6	NULL
7	7	NULL
8	8	NULL
9	9	NULL
10	10	Customer EEALV 8901 Tsawassen Blvd. 10111 Tsawas...
11	11	NULL
12	12	NULL
13	13	NULL
14	14	NULL
15	15	Customer JUWXK Av. dos Lusíadas, 6789 10087 Sao P...

Doen we deze query nu met CONCAT,

```
SELECT custid,
concat(companyname,space(1),address,space(1),postalcode,space(1),city,s
pace(1),region,space(1),country) as companyaddress
FROM sales.customers
```

dan krijgen we dit resultaat:

	custid	companyaddress
1	1	Customer NRZBB Obere Str. 0123 10092 Berlin Germany
2	2	Customer MLTDN Avda. de la Constitución 5678 10077 México D.F. Mexico
3	3	Customer KBUDE Mataderos 7890 10097 México D.F. Mexico
4	4	Customer HFBZG 7890 Hanover Sq. 10046 London UK
5	5	Customer HGVLZ Berguvsvägen 5678 10112 Luleå Sweden
6	6	Customer XHXJV Forsterstr. 7890 10117 Mannheim Germany
7	7	Customer QXVLA 2345, place Kléber 10089 Strasbourg France
8	8	Customer QUHWH C/ Araquil, 0123 10104 Madrid Spain
9	9	Customer RTXGC 6789, rue des Bouchers 10105 Marseille France
10	10	Customer EEALV 8901 Tsawassen Blvd. 10111 Tsawassen BC Canada
11	11	Customer UBHAU Fauntleroy Circus 4567 10064 London UK
12	12	Customer PSNMQ Cerrito 3456 10057 Buenos Aires Argentina
13	13	Customer VMLOG Sierras de Granada 7890 10056 México D.F. Mexico
14	14	Customer WNMAF Hauptstr. 0123 10065 Bern Switzerland
15	15	Customer JUWXK Av. dos Lusíadas, 6789 10087 Sao Paulo SP Brazil
16	16	Customer GYBBY Berkeley Gardens 0123 Brewery 10039 London UK

geen NULL waarden meer als companyaddress.

Ook sorteren met NULL waarden is interessant:

Het sorteren van NULL-waarden kan verschillen van DBMS tot DBMS.

Mogelijkheden:

1. De NULL-waarden komen altijd eerst, zowel bij ASC als DESC
2. De NULL-waarden komen altijd laatst, zowel bij ASC als DESC
3. **De NULL-waarden worden gezien als de kleinste waarden**
4. De NULL-waarden worden aanzien als de grootste waarden

Mogelijkheid 3 is geldig voor Microsoft SQL Server DBMS

ORDER BY clause

ORDER BY component

- Indien men niets specifieert, wordt een resultaat “oplopend” gesorteerd
- Oplopend sorteren kan ook expliciet aangegeven worden met de term “ASC” (ascending)
- Aflopend sorteren kan bekomen worden door “DESC” mee te geven (descending)
- Voor numerieke waarden, datums en tijden is de betekenis van op- en aflopend sorteren evident

ORDER BY component

Voor alfanumerieke waarden hangt dit af van de karakterset ....
A, B, C, ... of
A, a, B, b, ... of
A, a, AE, Ä, ä, …. of ...

Bij ASCII:
(blanco) ! “ # $ % & ‘ ( ) * + , - . / 0 1 2 3 4 5 6 7 8 9 : ; <
= > ? @ A B C D E F G H I J K L M N O P Q R S T U V
W X Y Z \ ^ _ `a b c d e f g h i j k l m n o p enz.

CASE Statement

Een CASE statement is een If .. Then .. Else statement in SQL termen. Deze statements zijn als volgt opgebouwd:

```
SELECT KOLOMNAAM, KOLOMNAAM2,
CASE
WHEN KOLOMNAAM3 = 'IETS' then 'IETS ANDERS'
WHEN KOLOMNAAM3 = 'IETS2' then 'IETS ANDERS2'
ELSE 'NOG IETS ANDERS'
END AS 'ALTERNATIEVE KOLOMNAAM'
FROM SCHEMANAAM.TABELNAAM
```

De WHEN is de IF/ELSEIF in dit verhaal.

Net zoals in C# is het mogelijk om meerdere condities in een WHEN vraag te zetten. Dit wordt in volgende query gedemonstreerd. Query:

```
SELECT custid, companyname, country,
case
when country = 'Poland' then 'Dzień dobry'
when country = 'Germany' then 'Guten Tag'
when country = 'France' then 'Bonjour'
when country = 'Canada' and region = 'Québec' then 'Bonjour'
when country = 'Canada' then 'Good day'
else 'Sorry, no correct translation'
end as Greeting
FROM sales.customers
order by custID desc
```

resulteert in de volgende data:

	custid	companyname	country	Greeting
1	91	Customer CCFIZ	Poland	Dzien dobry
2	90	Customer XBBVR	Finland	Sorry, no correct translation
3	89	Customer YBQTI	USA	Sorry, no correct translation
4	88	Customer SRQVM	Brazil	Sorry, no correct translation
5	87	Customer ZHYOS	Finland	Sorry, no correct translation
6	86	Customer SNXOJ	Germany	Guten Tag
7	85	Customer ENQZT	France	Bonjour
8	84	Customer NRCSK	France	Bonjour
9	83	Customer ZRNDE	Denmark	Sorry, no correct translation
10	82	Customer EYHKM	USA	Sorry, no correct translation
11	81	Customer YQQWW	Brazil	Sorry, no correct translation
12	80	Customer VONTK	Mexico	Sorry, no correct translation
13	79	Customer FAPSM	Germany	Guten Tag
14	78	Customer NLTYP	USA	Sorry, no correct translation
15	77	Customer LCYBZ	USA	Sorry, no correct translation
16	76	Customer SFOGW	Belgium	Sorry, no correct translation

Kijken we alleen naar Canada:

	custid	companyname	country	Greeting
1	51	Customer PVDZC	Canada	Bonjour
2	42	Customer IAIJK	Canada	Good day
3	10	Customer EEALV	Canada	Good day